

Train / test split

	Features	Class
Training	X1	1
	X2	2
	X3	3
	X4	1
	X5	2
	X6	3
	X7	1
Test	X8	3
	X9	2

Test with unseen instances

Query	Doc	Rel	
news	Doc1	5	Training queries
news	Doc2	3	
news	Doc3	2	
cats	Doc4	1	
cats	Doc5	4	
dogs	Doc4	3	
dogs	Doc6	2	
football	Doc3	4	Test queries
football	Doc9	2	

Test with unseen **queries**

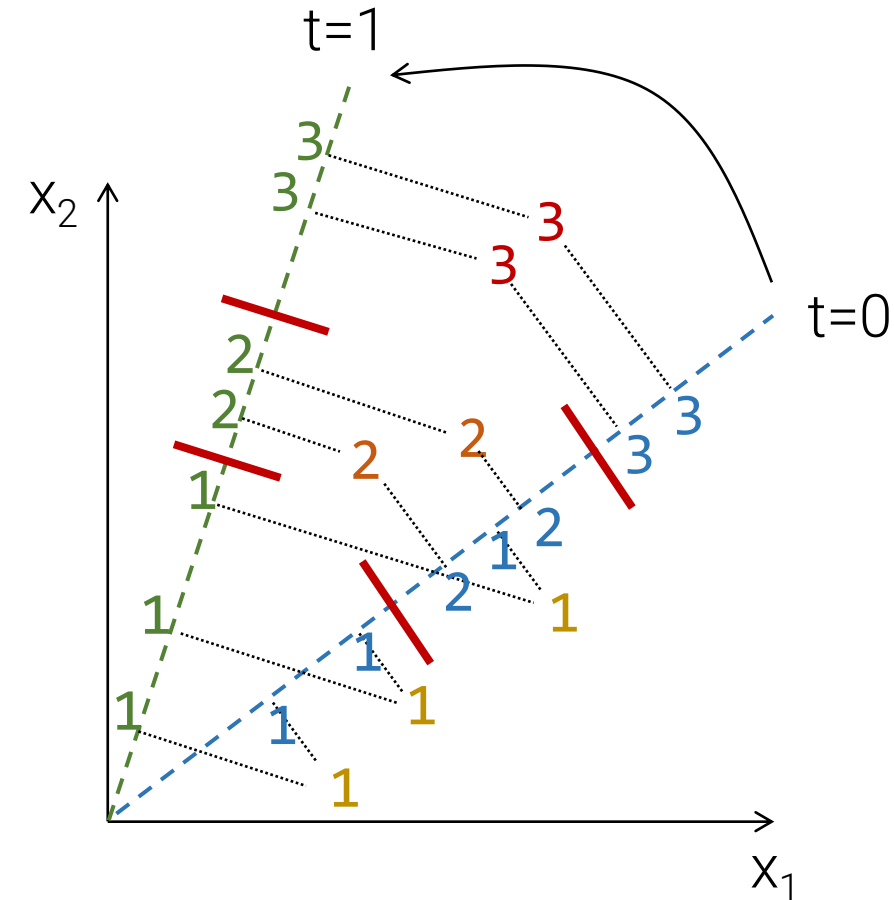
Features

		Query-dependent features		Query-only features	Doc-only features		
		Query ID	Title BM25	Body BM25	Query length	Doc PageRank	Rel
Keep query logical grouping	1	12.3	3.4	3	0.012	1	
	1	15.1	6.3	3	0.026	0	
	2	2.2	4.3	5	0.042	1	
	2	14.8	5.7	5	0.021	0	
	3	3.1	10.4	2	0.035	1	
	3	9.4	7.3	2	0.121	0	
	4	6.2	9.0	2	0.091	0	
	4	10.8	9.1	8	0.036	1	

Actual queries and documents are not needed here

Ordinal regression with Prank (Perceptron Ranking)

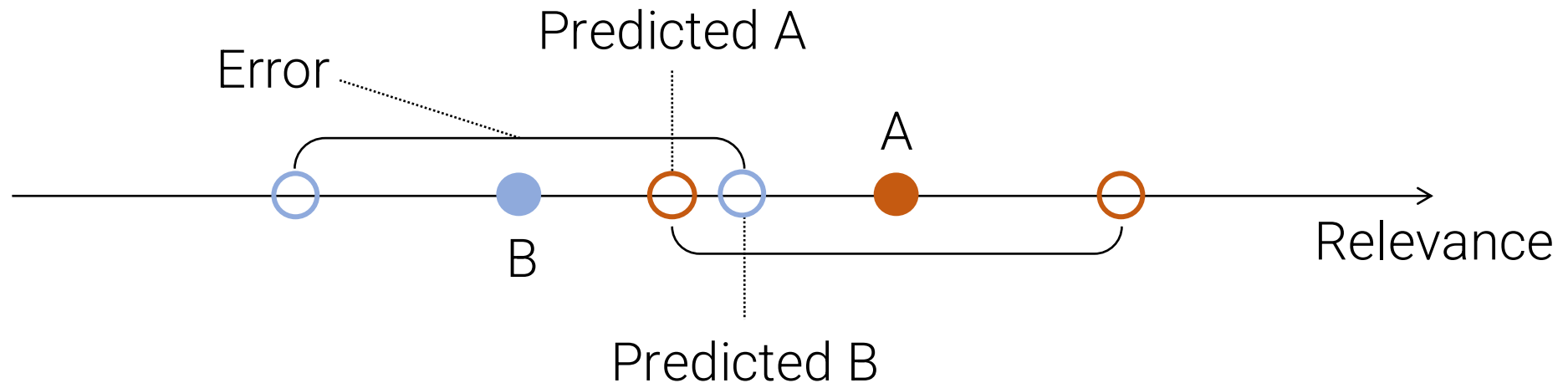
- K ordered labels $\{1, 2, \dots, K\}$
- Initialize a random hyperplane
- Project points on the hyperplane
- Select K-1 thresholds on the projected hyperplane to identify K classes
- For each misclassification, update the weights
- Project on the new hyperplane
- Repeat until convergence



Shortcomings of pointwise approaches

Pointwise accuracy $\neq$ ranking accuracy

A small absolute error can still lead to ranking misclassifications



$A > B$

$\text{Predicted } A < \text{Predicted } B$

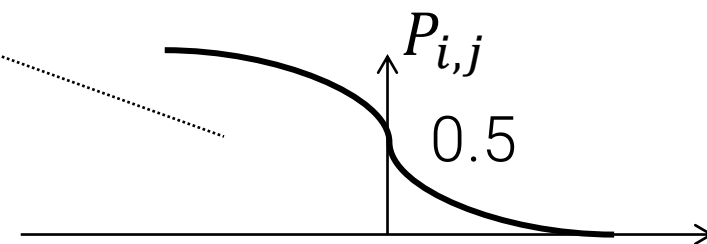
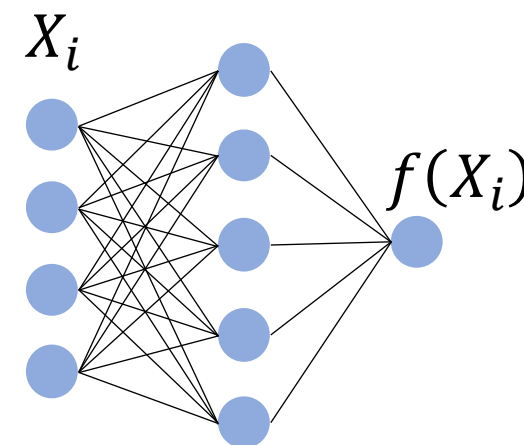
Pairwise LTR with RankNet

$f(X_i)$ is the score, estimated using a shallow, fully connected neural network, using gradient descent

- For each query q and its documents $D_q = \{d_1, \dots, d_m\}$
- Consider all pairs $d_i, d_j \in D_q$
- Target label $y_{i,j} = \begin{cases} 1 & \text{if } i \triangleright j \\ 0 & \text{otherwise} \end{cases}$ d_i is ranked higher than d_j
- Modeled probability

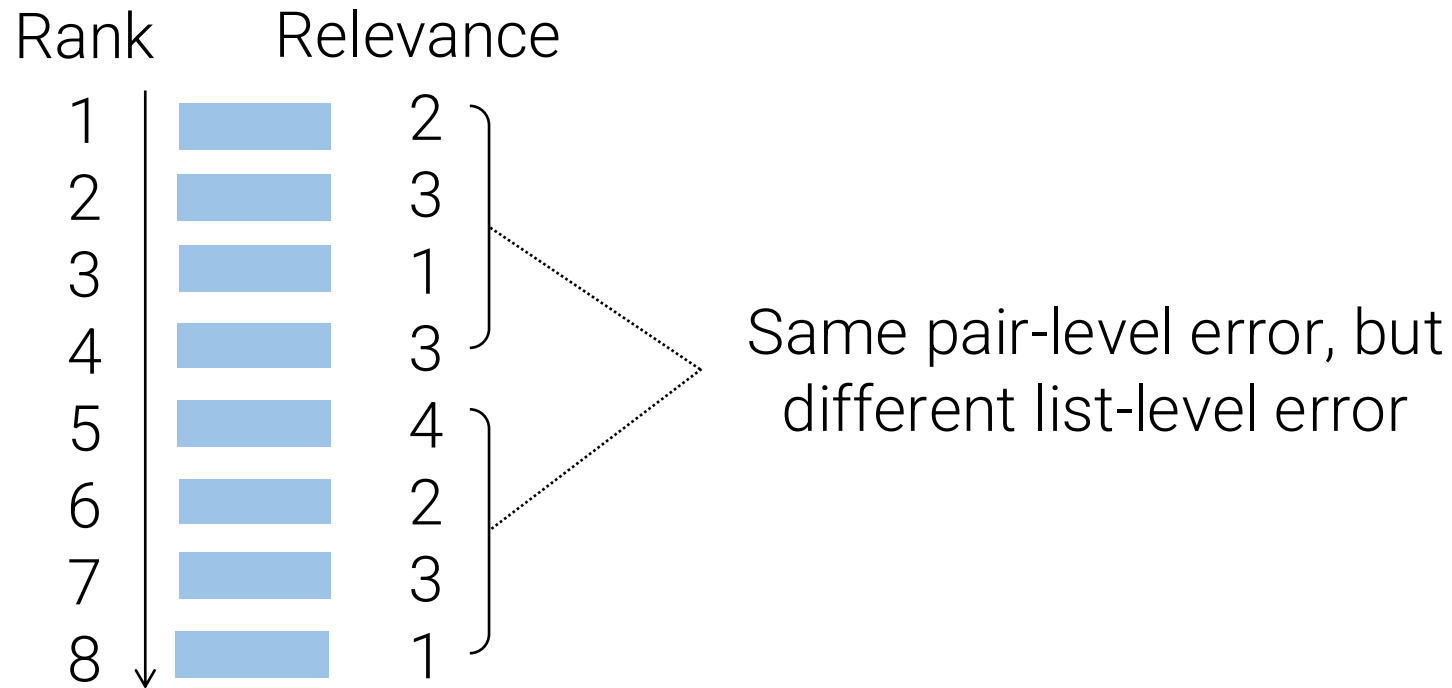
$$P_{ij} = P(d_i \triangleright d_j | q) = \frac{\exp(f(X_i) - f(X_j))}{1 + \exp(f(X_i) - f(X_j))}$$

- Optimize Cross Entropy loss
 - $\mathcal{L}_{ij} = -y_{ij} \log(P_{ij}) - (1 - y_{ij}) \log(1 - P_{ij})$
- Gradient to update network weights
 - $\lambda_{ij} = \frac{\delta(\mathcal{L}_{ij})}{\delta(f(X_i))} = \sigma(f(X_i) - f(X_j)) - y_{ij}$



Shortcoming of pairwise approaches

- Document pairs have the same weight
 - Errors for pairs ranked at the top of the result list should weigh more



Listwise LTR with ListNet

- Training loss is directly based on the difference between the prediction list and the ground truth list
- Target: directly optimize the ranking evaluation measures

PL (Plackett-Luce) probability model: documents with high scores tend to be ranked at the top of the list

$$P_f([j_1, \dots, j_m]) = \prod_{i=1}^m \frac{e^{f(X_i)}}{\sum_{l=i}^m e^{f(X_l)}}$$

s_{X_i}

- s_{X_i} is estimated using a neural network
- Loss can be any function to compare real ranking and predicted ranking (cross entropy, NDCG, etc.)

Probability of permutation A,B,C

$$P([A, B, C]) = \frac{s_A}{s_A + s_B + s_C} \cdot \frac{s_B}{s_B + s_C} \cdot \frac{s_C}{s_C}$$

Sum of all permutations probabilities = 1

LambdaRank (pairwise and listwise hybrid)

- Variation on RankNet
- For each pair of documents d_i, d_j , calculate $\Delta_{i,j}(NDCG)$ when swapping them on the list
- The gradient λ is defined as a function of $\Delta_{i,j}(NDCG)$
 - $\lambda_{ij} = \frac{-\Delta_{ij}(NDCG)}{1 + \exp(f(X_i) - f(X_j))}$, where i is ranked higher than j in the ground truth
- Approach is still pairwise, but incorporates list-wise metric for optimization

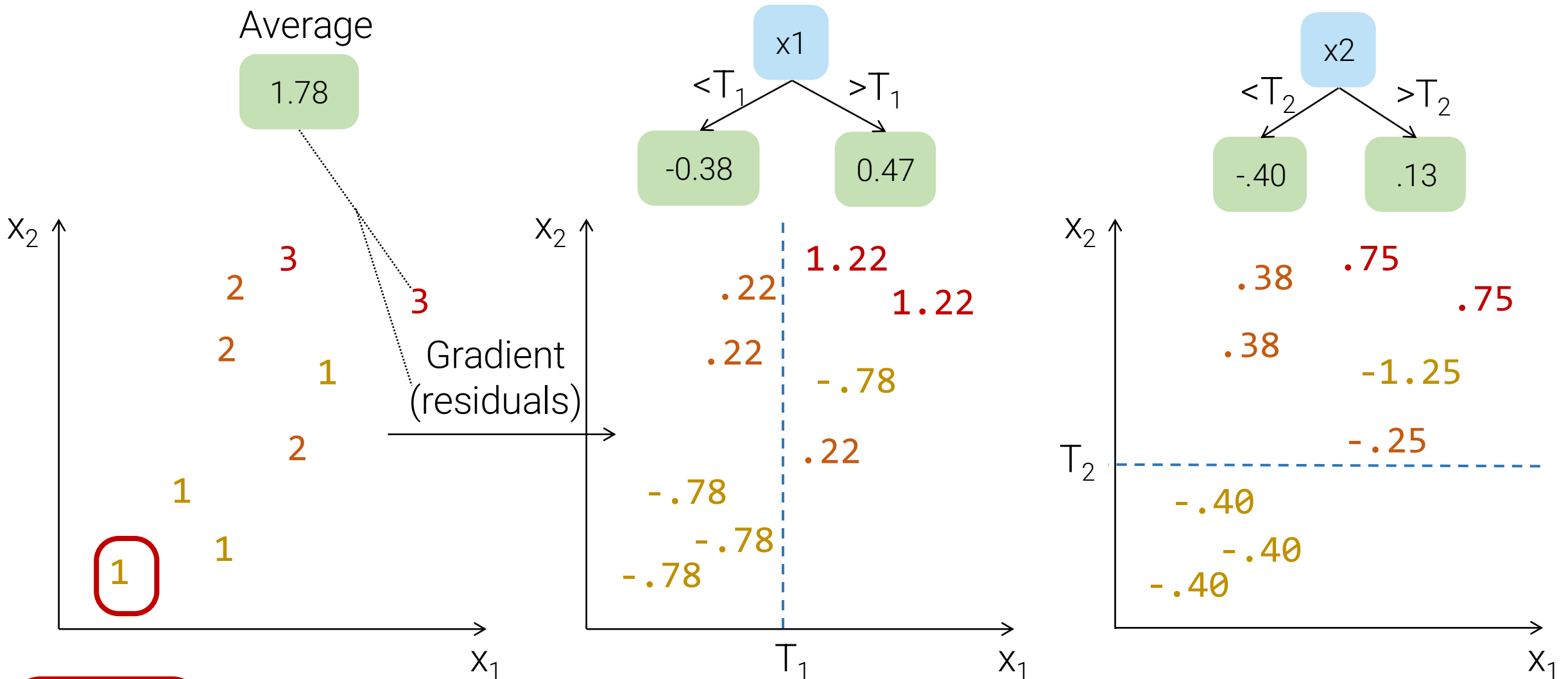
LambdaMART

- Combine LambdaRank with regression trees
- MART: Multiple Additive Regression Trees
 - Also known as Gradient Boosting Regression Trees (GBRT)

Gradient boosting

- Ensemble of weak learners, each learning from the errors of previous ones
- $\hat{y} = F_{m+1}(x) = F_m(x) + h_m(x)$
- Start with $F_1(x) = h_1(x) = \bar{y}$ ----- Average of target variable in training set
- Get $r_{1,i} = \frac{\partial \mathcal{L}(y_i, \hat{y}_i)}{\partial \hat{y}_i}$ ----- Derivative of loss function in my estimate is the gradient
- Train a weak learner to predict $r_{1,i}$
- Get new gradients and repeat

Gradient Boosting Regression Trees



Prediction = $1.78 + \lambda \cdot -0.38 + \lambda \cdot -0.40 = 1.78 + 0.3 \cdot -0.38 + 0.3 \cdot -0.40 = 1.546$

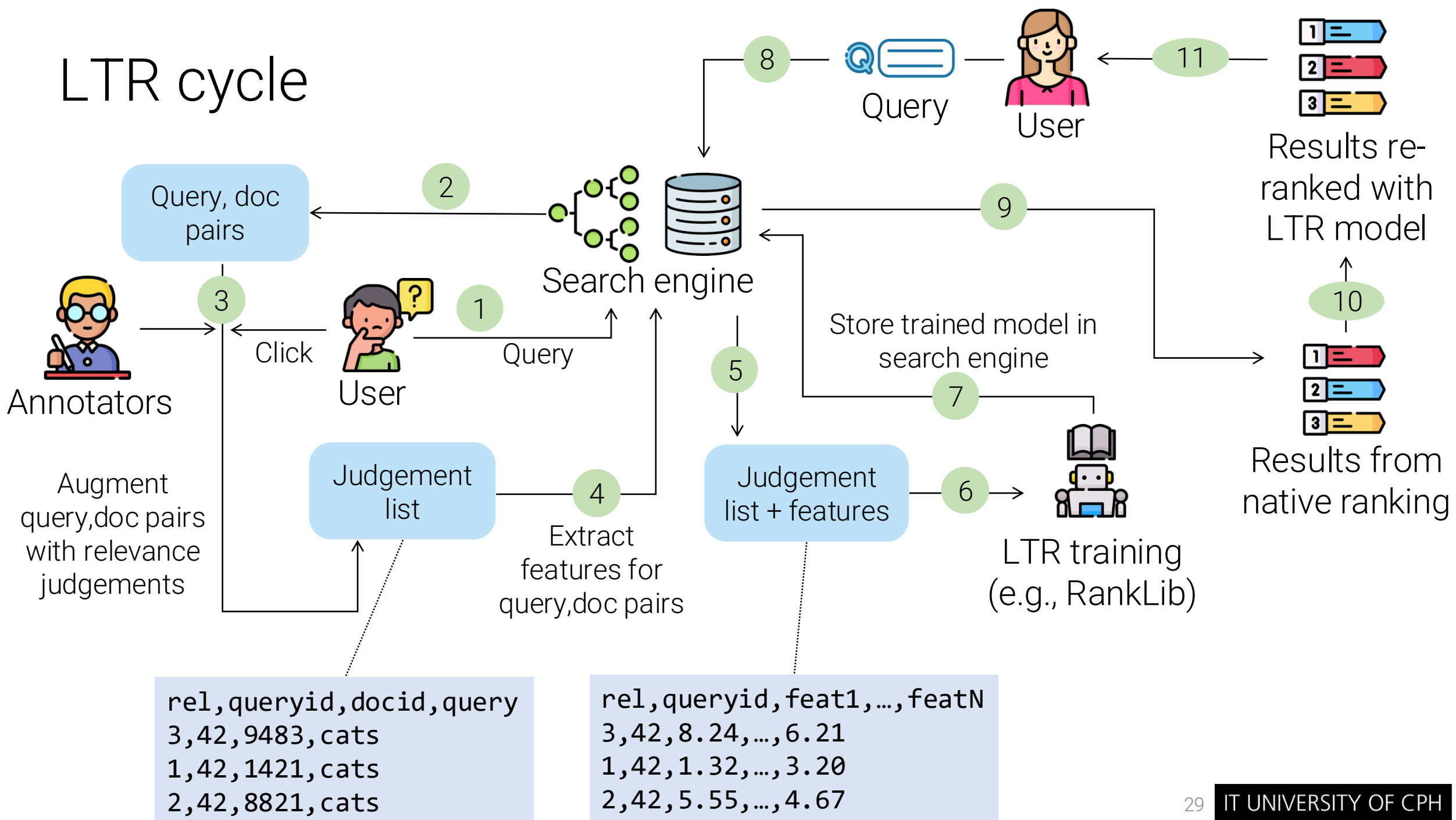
LambdaMART

GBRT where the gradient is defined as λ_i

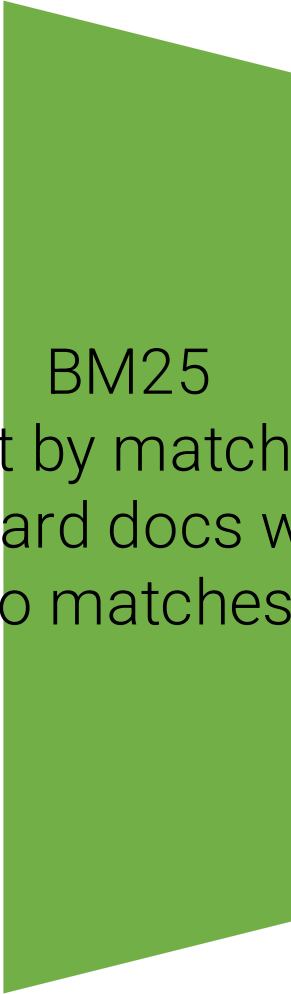
$$\lambda_{ij} = \frac{-\Delta_{i,j}(NDCG)}{1 + \exp(\hat{y}_i - \hat{y}_j)} \quad \left. \vphantom{\lambda_{ij}} \right\} \text{Gradient for pair } i, j \text{ such that } i \text{ is ranked higher than } j \text{ in the ground truth}$$

$$\lambda_i = \sum_{i,j} \lambda_{ij} - \sum_{k,i} \lambda_{ki}$$

LTR cycle



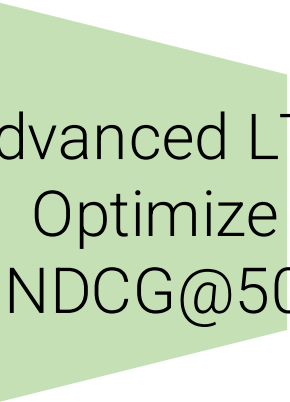
Model chaining



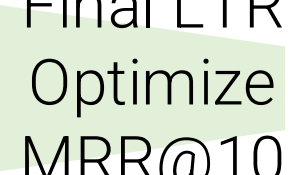
BM25
Sort by matches
Discard docs with
no matches



Simple LTR
Optimize
P@100



Advanced LTR
Optimize
NDCG@50



Final LTR
Optimize
MRR@10

Refine the top K ranking with iterations of different models.
As K gets smaller, models used are slower and optimize for different metrics

Install LTR Elasticsearch plugin

- Developed by Opensource Connections, Snagajob, Wikimedia Foundation
 - Still under development, but widely used (e.g., AWS)
 - Run when ES is not running
 - Mind the version!
-
- NOT needed if you have installed the custom Docker-based ES from our TAs

```
# install ES LTR plugin
./elasticsearch-8.15.3/bin/elasticsearch-plugin install
https://github.com/o19s/elasticsearch-learning-to-rank/releases/download/v1.5.10-es8.15.5/ltr-plugin-v1.5.10-es8.15.5.zip
```

<https://elasticsearch-learning-to-rank.readthedocs.io/en/latest/index.html>

<https://github.com/o19s/elasticsearch-learning-to-rank/releases>

<https://docs.aws.amazon.com/opensearch-service/latest/developerguide/learning-to-rank.html>

What the LTR plugin does

- Feature store
 - Allows user to define features and store them
 - Multiple stores could be created
- Model store
 - Can import LTR models that are trained with third-party tools (mainly RankLib)
- Result (re)ranking
 - At query time, a model can be selected to (re)rank results
- It is NOT a tool to train LTR models

Install RankLib

- A Java library implementing most popular LTR algorithms
- Not well-established python wrappers (AFAIK)
- Just download .jar file and place it in the directory where you are executing your code
- You can execute it from python using

```
import os  
os.system('java -jar RankLib-2.9.jar ...params...')
```

- You need a JDK to run it (if you use OpenJDK, you need v.11)

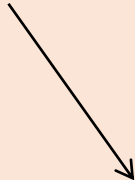
<https://sourceforge.net/projects/lemur/files/lemur/RankLib-2.18/>

A note on templated queries

Save templates and instantiate them later

```
# Define and save a template
PUT _scripts/example-template
{
  "script": {
    "lang": "mustache",
    "source": {
      "query": {
        "match": {
          "product_description": "{{query_string}}"
        }
      },
      "size": "{{size}}"
    }
  }
}
```

“Mustache” notation



```
# Validate a search template
POST _render/template
{
  "id": "example-template",
  "params": {
    "query_string": "circular saw",
    "size": 5
  }
}
```

```
# Run a template query
GET my-index/_search/template
{
  "id": "example-template",
  "params": {
    "query_string": "circular saw",
    "size": 5
  }
}
```

<https://www.elastic.co/guide/en/elasticsearch/reference/current/search-template.html>
<https://mustache.github.io/mustache.5.html>

Steps:

1. Index the data (as seen previously)
2. Create a feature store
3. Define a feature set
4. Query the index to get feature for the query,doc pairs in the judgement list
5. Use the feature file to train a model in RankLib
6. Load the RankLib model file into ES
7. Formulate queries for re-ranking results according to model

Step-by-step guide by Elasticsearch [warmly recommended read!]:

<https://elasticsearch-learning-to-rank.readthedocs.io/en/latest/index.html>

2,3. Create feature store and define features

```
# Initialize feature store  
PUT _ltr  
DELETE _ltr # delete it
```

```
# Get features info  
GET _ltr/_featureset/  
  
GET _ltr/_featureset/  
depot_textual_features  
  
# Delete feature set  
DELETE _ltr/_featureset/  
depot_textual_features
```

We want to create two features, one per text field

```
# Register a feature list  
POST _ltr/_featureset/depot_textual_features  
{  
  "featureset": {  
    "name": "depot_textual_features",  
    "features": [  
      {  
        "name": "f_product_title",  
        "params": [  
          "keywords"  
        ],  
        "template_language": "mustache",  
        "template": {  
          "match": {  
            "product_title": "{{keywords}}"  
          }  
        }  
      }, ... # add more features to the list here  
    ]  
  }  
}
```

Feature set names and feature names must be unique

4. Query index to get features

The tricky/boring step, several ways of doing it. This is what I did.

1. From relevance table, create a dictionary structure similar to:

```
{Queryid: {DocId: {"relevance": relevance_score}}}
```

2. Submit queries containing query and list of its documents (see next slide)
3. Parse the result and augment the dictionary

```
{Queryid: {DocId: {"relevance": relevance_score, "featurename1":  
feature_value1, "featurename2": feature_value2}}
```

4. Convert the dictionary into a RankLib file format

4. Query index to get features

Query the index for query-doc features

POST homedepot/_search

```
{
  "query": {
    "bool": {
      "filter": [
        {"terms": {"_id": LIST_OF_PIDS_HERE}},
        {
          "sltr": {
            "_name": "logged_featureset",
            "featureset": "depot_textual_features",
            "params": {"keywords": QUERY_HERE}
          }
        }
      ]
    },
    "ext": {
      "ltr_log": {
        "log_specs": {
          "name": "log_entry",
          "named_query": "logged_featureset"
        }
      }
    }
  }
}
```

Feature set name that
we defined in step 3



5. Use the feature file to train a model in RankLib

Train a LambdaMART model optimized for NDCG

```
import os
```

```
cmd = f'java -jar RankLib-2.9.jar -train {feature_file}  
      -ranker 6 -metric2t NDCG@5 -save {model_file}'
```

```
os.system(cmd)
```

Feature file (no header)

2.67	qid:1	1:6.4248157	2:4.00418
2.33	qid:1	1:3.6470122	2:7.31683
3.0	qid:1	1:4.2923274	2:7.75500
3.0	qid:2	1:10.244193	2:15.8570
3.0	qid:2	1:11.218935	2:4.49711
2.33	qid:2	1:11.161285	2:4.97494
2.0	qid:3	1:5.759474	2:6.07054
2.33	qid:3	1:6.0090837	2:5.35044
2.0	qid:3	1:10.908635	2:7.41712
2.67	qid:3	1:5.755543	2:9.81112
2.0	qid:3	1:4.914073	2:10.2127
2.0	qid:3	1:5.0958967	2:7.45944
2.33	qid:3	1:4.934701	2:5.66462
1.67	qid:3	1:6.7630534	2:6.13581
2.0	qid:3	1:11.3838215	2:10.3739
2.67	qid:4	1:8.639042	2:9.60723
3.0	qid:4	1:3.640378	2:4.03868

Relevance

Query ID

Feature1:value1

6. Load trained model into ES

```
POST _ltr/_featureset/depot_textual_features/_createmodel
{
  "model": {
    "name": "ndcg_model", #arbitrary name
    "model": {
      "type": "model/ranklib",
      "definition": ...all text from model file here...
    }
  }
}
```


7. Formulate queries for re-ranking results

```
# Rank results with trained model  
POST homedepot/_search  
{  
  "query": {  
    "sltr": {  
      "params": {  
        "keywords": "circular saw"  
      },  
      "model": "ndcg_model"  
    }  
  }  
}
```

Custom name we gave
to the trained model



This command applies the model to all documents and ranks them accordingly

Very resource intensive

In practice, rank first by default scoring, then rerank the top K with the model

7. Formulate queries for re-ranking results

Rank results with trained model

POST homedepot/_search

```
{  
  "query": { ←  
    "match": {  
      "_all": "circular saw"  
    }  
  },  
  "rescore": { ←  
    "window_size": 50,  
    "query": {  
      "rescore_query": {  
        "sltr": {  
          "params": {  
            "keywords": "circular saw"  
          },  
          "model": "ndcg_model"  
        }  
      }  
    }  
  }  
}
```

Execute a query and
ranks with standard
BM25

Rescore top 50 results
selected by BM25
using the trained model

Dive in!

- In addition:
 - How would the LTR model perform on a train-test split or on cross validation?
 - What if you had to add some static relevance signal? (e.g., product quality)
 - How do BM25 parameters affect the different features?
 - Can you compute the exact NDCG@K for a given query and ranked document list (with assigned relevance scores)
 - Standard IR metrics are implemented in sklearn